Python Engineering

WEEK 03 - Modules & Coordinates | CLASS 11 - Coordinate Transforms | EX19

Objetivo: introducir transformaciones de coordenadas mediante matrices de rotacion. Trabajaremos primero en 2D para entender bien la geometria, el producto matricial y la transformacion inversa.

Ruta

```
01_python_bootcamp/
`-- week_03_modules_and_coordinates/
    `-- class_11_coordinate_transforms/
        `-- exercises/
            `-- ex19_coordinate_rotation.py
```

Convencion del ejercicio

Usaremos una **rotacion activa**: el sistema de coordenadas permanece fijo y rotamos el vector. Un angulo positivo rota en sentido antihorario (counter-clockwise).

Valores de referencia

Caso	Vector	Angulo	Resultado esperado
A	[100.0, 0.0]	90 deg	[0.0, 100.0] aprox.
B	[3.0, 4.0]	30 deg	[0.5981, 4.9641] aprox.
Norma caso B	||[3,4]||	-	5.0 antes y despues

BLOCK A - Convertir grados a radianes

```
angle_deg = 90.0
angle_rad = np.deg2rad(angle_deg)
```

BLOCK B - Crear la matriz de rotacion

```
def rotation_matrix_2d(angle_deg):
    theta = np.deg2rad(angle_deg)

    R = np.array([
        [ np.cos(theta), -np.sin(theta)],
        [ np.sin(theta),  np.cos(theta)]
    ])

    return R
```

Comprueba shape, ndim y determinantemente observa que R tiene shape (2,2).

BLOCK C - Rotar un vector

```
def rotate_vector_2d(vector, angle_deg):
    R = rotation_matrix_2d(angle_deg)
    rotated = R @ vector
    return rotated
```

El operador @ realiza producto matricial. Con R de shape (2,2) y vector de shape (2,), el resultado tiene shape (2,).

BLOCK D - Caso de referencia A

```
vector_A = np.array([100.0, 0.0])
rotated_A = rotate_vector_2d(vector_A, 90.0)

# Expected approximately:
# [0.0, 100.0]
```

No exijas que el primer valor sea exactamente cero: con coma flotante puede aparecer un numero muy pequeno como 6e-15.

BLOCK E - Norma del vector

```
norm_before = np.linalg.norm(vector_A)
norm_after = np.linalg.norm(rotated_A)

same_norm = np.allclose(norm_before, norm_after)
```

Una rotacion ideal cambia la direccion, pero no la longitud del vector.

BLOCK F - Transformacion inversa

```
R = rotation_matrix_2d(30.0)
vector_B = np.array([3.0, 4.0])

rotated_B = R @ vector_B
recovered_B = R.T @ rotated_B
```

Para una matriz de rotacion ortogonal, la inversa coincide con la traspuesta: R^-1 = R.T. Compruebalo con np.allclose(recovered_B, vector_B).

BLOCK G - Dos rotaciones consecutivas

```
v1 = rotate_vector_2d(vector_B, 20.0)
v2 = rotate_vector_2d(v1, 40.0)

v_total = rotate_vector_2d(vector_B, 60.0)

np.allclose(v2, v_total)
```

Para rotaciones 2D con la misma convencion, 20 deg seguidos de 40 deg equivalen a una rotacion total de 60 deg.

BONUS - Rotacion 3D alrededor de Z

```
def rotation_matrix_z(angle_deg):
    theta = np.deg2rad(angle_deg)
    return np.array([
        [np.cos(theta), -np.sin(theta), 0.0],
        [np.sin(theta),  np.cos(theta), 0.0],
        [0.0,            0.0,           1.0]
    ])

vector_3d = np.array([10.0, 0.0, 5.0])
rotated_3d = rotation_matrix_z(90.0) @ vector_3d
# expected approx [0.0, 10.0, 5.0]
```

VALIDACION FINAL

```
Explain in comments:
- What does np.deg2rad() receive and return?
- What are the shape and meaning of R in 2D?
- Why do we use @ instead of * for R and vector?
- Why should vector norm remain unchanged after a rotation?
- What does R.T mean?
- Why can R.T recover the original vector?
- Why can a theoretical zero appear as a tiny non-zero float?
- What is the difference between rotating a vector and changing the coordinate frame?
```

Criterio de cierre

EX19 estara completado cuando puedas construir una matriz de rotacion 2D, aplicarla con @, comprobar la conservacion de norma y recuperar el vector original mediante la transformacion inversa.